

THUẬT TOÁN TÌM KIẾM CỤC BỘ (LOCAL SEARCH ALGORITHMS)

CHƯƠNG 4, PHẦN 3–4

Phác thảo

- ◇ Leo đồi
- ◇ Ủ mô phỏng
- ◇ Thuật toán di truyền (tóm lược)
- ◇ Tìm kiếm cục bộ trong không gian liên tục (rất ngắn gọn)

Thuật toán cải tiến lặp lại

Trong nhiều vấn đề tối ưu hóa, **path** không liên quan;
bản thân trạng thái mục tiêu là giải pháp

Khi đó không gian trạng thái = tập hợp các cấu hình “hoàn thành”;

tìm cấu hình **tối ưu**, ví dụ: TSP

hoặc tìm cấu hình thỏa mãn các ràng buộc, ví dụ: thời gian biểu

Trong những trường hợp như vậy, có thể sử dụng thuật toán cải tiến lặp lại;
giữ một trạng thái “hiện tại” duy nhất, cố gắng cải thiện nó

Không gian cố định, thích hợp cho việc tìm kiếm trực tuyến cũng như ngoại tuyến

Ví dụ: Vấn đề của nhân viên bán hàng khi đi du lịch

Bắt đầu với bất kỳ chuyến tham quan hoàn chỉnh nào, thực hiện trao đổi theo cặp

Các biến thể của phương pháp này đạt được trong khoảng 1% mức tối ưu rất nhanh chóng với hàng ngàn thành phố

Ví dụ: n -quân hậu

Đặt quân hậu n lên bàn $n \times n$ không có hai quân hậu trên cùng hàng, cột hoặc đường chéo

Di chuyển một nữ hoàng để giảm số lượng xung đột

Hầu như luôn giải quyết được các vấn đề của n -nữ hoàng gần như ngay lập tức

cho n rất lớn, ví dụ: $n = 1\text{million}$

Leo đồi (hoặc lên/xuống dốc)

"Giống như leo Everest trong sương mù dày đặc với chúng mất trí nhớ"

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
inputs: problem, a problem
local variables: current, a node
                   neighbor, a node

current ← MAKE-NODE(INITIAL-STATE[problem])
loop do
  neighbor ← a highest-valued successor of current
  if VALUE[neighbor] ≤ VALUE[current] then return STATE[current]
  current ← neighbor
end
```

Leo đòi tiếp.

Hữu ích khi xem xét cảnh quan không gian trạng thái

Leo đòi khởi động lại ngẫu nhiên vượt qua cực đại cục bộ—hoàn thành tầm thường

Di chuyển ngang ngẫu nhiên thoát khỏi vai vòng lặp trên cực đại

phẳng

Ủ mô phỏng

Ý tưởng: thoát khỏi cực đại địa phương bằng cách cho phép một số chuyển động “xấu”

nhưng giảm dần kích thước và tần số của chúng

function SIMULATED-ANNEALING(*problem*, *schedule*) **returns** a solution state

inputs: *problem*, a problem
schedule, a mapping from time to “temperature”

local variables: *current*, a node
next, a node
T, a “temperature” controlling prob. of downward steps

current $\leftarrow$ MAKE-NODE(INITIAL-STATE[*problem*])

for *t* $\leftarrow$ 1 **to** ∞ **do**

T $\leftarrow$ *schedule*[*t*]

if *T* = 0 **then return** *current*

next $\leftarrow$ a randomly selected successor of *current*

$\Delta E \leftarrow$ VALUE[*next*] – VALUE[*current*]

if $\Delta E > 0$ **then** *current* $\leftarrow$ *next*

else *current* $\leftarrow$ *next* only with probability $e^{\Delta E/T}$

Tính chất của quá trình ủ mô phỏng

Ở “nhiệt độ” cố định T , xác suất chiếm đóng trạng thái đạt phân phối Boltzman

$$p(x) = \alpha e^{\frac{E(x)}{kT}}$$

T giảm đủ chậm $\implies$ luôn đạt trạng thái tốt nhất x^*
vì $e^{\frac{E(x^*)}{kT}} / e^{\frac{E(x)}{kT}} = e^{\frac{E(x^*) - E(x)}{kT}} \gg 1$ dành cho T nhỏ

Đây có phải là một sự đảm bảo thú vị??

Được phát minh bởi Metropolis và cộng sự, 1953, để mô hình hóa quy trình vật lý

Được sử dụng rộng rãi trong bố trí VLSI, lập lịch trình hàng không, v.v.

Tìm kiếm chùm tia cục bộ

Idea: giữ trạng thái k thay vì 1; chọn top k trong số tất cả những người kế nhiệm của họ

Không giống như các tìm kiếm k chạy song song!

Các tìm kiếm tìm thấy trạng thái tốt tuyển dụng các tìm kiếm khác tham gia cùng họ

Sự cố: khá thường xuyên, tất cả các trạng thái k đều kết thúc trên cùng một ngọn đồi địa phương

Idea: chọn ngẫu nhiên k người kế nhiệm, thiên về những người giỏi

Hãy quan sát sự tương tự gần gũi với chọn lọc tự nhiên!

Thuật toán di truyền

= tìm kiếm chùm cục bộ ngẫu nhiên + tạo các trạng thái kế tiếp từ **cặp** trạng thái

Thuật toán di truyền tiếp.

GA yêu cầu các trạng thái được mã hóa dưới dạng chuỗi (GPs sử dụng programs)

Crossover giúp **iff chuỗi con là các thành phần có ý nghĩa**

Sự tiến hóa của GA $\neq$: ví dụ: gen thực mã hóa bộ máy sao chép!

Không gian trạng thái liên tục

Giả sử chúng tôi muốn xác định ba sân bay ở Romania:

- Không gian trạng thái 6-D được xác định bởi $(x_1, y_1), (x_2, y_2), (x_3, y_3)$
- hàm mục tiêu $f(x_1, y_1, x_2, y_2, x_3, y_3) =$

tổng bình phương khoảng cách từ mỗi thành phố đến sân bay gần nhất

Phương pháp rời rạc hóa biến không gian liên tục thành không gian rời rạc, ví dụ: gradient theo kinh nghiệm xem xét sự thay đổi của $\pm\delta$ trong mỗi tọa độ

Phương pháp tính toán Gradient

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial y_1}, \frac{\partial f}{\partial x_2}, \frac{\partial f}{\partial y_2}, \frac{\partial f}{\partial x_3}, \frac{\partial f}{\partial y_3} \right)$$

để tăng/giảm f , ví dụ: bằng $\mathbf{x} \leftarrow \mathbf{x} + \alpha \nabla f(\mathbf{x})$

Đôi khi có thể giải chính xác $\nabla f(\mathbf{x}) = 0$ (ví dụ: với một thành phố).

Newton–Raphson (1664, 1690) lặp lại $\mathbf{x} \leftarrow \mathbf{x} - \mathbf{H}_f^{-1}(\mathbf{x}) \nabla f(\mathbf{x})$

để giải $\nabla f(\mathbf{x}) = 0$, trong đó $\mathbf{H}_{ij} = \partial^2 f / \partial x_i \partial x_j$